Fraudulent Transaction Detection

WISIT SUWANNAO 67070501042

Overview

ธุรกรรมการเงินที่เป็นการฉ้อโกง (Fraud) ถือเป็นภัยคุกคามสำคัญต่อระบบการเงินทั่วโลก ส่งผลกระทบทั้งต่อสถาบันการเงินและผู้บริโภค การตรวจจับ Fraud แบบ Real-Time จึงมีความสำคัญอย่างยิ่งในการป้องกันความเสียหายที่อาจเกิดขึ้น

Dataset

Dataset ที่ใช้ในการแข่งขันนี้เป็นข้อมูลธุรกรรมการเงินที่จำลองมาจาก **PaySim** ซึ่งสร้างขึ้นโดยอิงจากพฤติกรรมของธุรกรรมจริงในระบบ Mobile Money โดยมีรายละเอียดดังนี้

Column	Description
`time_ind`	หน่วยเวลาจำลอง (1 Step = 1 ชั่วโมง, รวม 744 Steps = 30 วัน)
`transac_type`	ประเภทธุรกรรม: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER
`amount`	จำนวนเงินในธุรกรรม (สกุลเงินท้องถิ่น)
`src_acc`	บัญชีต้นทาง (ผู้ริเริ่มธุรกรรม)
`src_bal`	ยอดคงเหลือของบัญชีต้นทาง **ก่อน** ธุรกรรม
`src_new_bal`	ยอดคงเหลือของบัญชีต้นทาง **หลัง** ธุรกรรม
`dst_acc`	บัญชีปลายทาง (ผู้รับเงิน)
`dst_bal`	ยอดคงเหลือของบัญชีปลายทาง **ก่อน** ธุรกรรม (ว่างเปล่าสำหรับ Merchant)
`dst_new_bal`	ยอดคงเหลือของบัญชีปลายทาง **หลัง** ธุรกรรม (ว่างเปล่าสำหรับ Merchant)
`is_fraud`	**Target Variable** — 1 หากธุรกรรมเป็น Fraud, 0 หากไม่ใช่
`is_flagged_fraud`	Flag จากระบบ Rule-Based (ธุรกรรม TRANSFER มูลค่าเกิน 200,000)

ข้อสังเกตสำคัญ: จากการวิเคราะห์ Dataset พบว่า Fraud เกิดขึ้น **เฉพาะในประเภท TRANSFER และ CASH-OUT เท่านั้น** และมักมีพฤติกรรมโอนเงินจนบัญชีต้นทางเหลือยอดเป็น 0 ซึ่งเป็น Pattern หลักที่ใช้ในการออกแบบ Feature

Pipeline Architecture

1. **Exploratory Data Analysis (EDA)** วิเคราะห์การกระจายตัวของ Class (Fraud vs. Legitimate), ประเภทธุรกรรม, ค่าว่าง และสถิติพื้นฐานของ Dataset เพื่อทำความเข้าใจโครงสร้างข้อมูลก่อนเริ่มสร้างโมเดล
2. **Data Imputation & Feature Engineering** เติมค่าว่างในคอลัมน์ Balance ของ Merchant และสร้าง Feature ใหม่กว่า 30 ตัวจาก Domain Knowledge เช่น ความคลาดเคลื่อนของ Balance (Accounting Error), Zero-Drain Flags, Log Transforms, Ratio Features และ Account-Level Aggregation
3. **Ensemble Machine Learning (XGBoost + Random Forest)** ฝึกโมเดลสองตัวด้วย 5-Fold Stratified Cross-Validation โดยเก็บ Out-of-Fold (OOF) Predictions จากทั้งสองโมเดลเพื่อนำไปทำ Weighted Soft Voting Ensemble (XGBoost 65% + Random Forest 35%)
4. **High-Resolution Threshold Optimization** ค้นหา Decision Threshold ที่ให้ F1 Macro Score สูงที่สุดโดยทดสอบทุก Step ที่ 0.001 ในช่วง 0.001–0.999 เทียบกับ OOF Predictions เพื่อหลีกเลี่ยง Data Leakage

1. Import Dependencies

นำเข้า Library ที่ใช้ในการวิเคราะห์ข้อมูล สร้างโมเดล และประเมินผล ได้แก่ `pandas` / `numpy` สำหรับ Data Manipulation, `matplotlib` / `seaborn` สำหรับ Visualization, `scikit-learn` สำหรับ Cross-Validation และ Random Forest และ `xgboost` สำหรับ Gradient Boosting

```
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

2. Path Configuration

กำหนด Path ของไฟล์ข้อมูลจากตำแหน่ง Dataset

```
train_path  = "/kaggle/input/competitions/fraud-detection-cpe-232-data-models/train.csv"
test_path   = "/kaggle/input/competitions/fraud-detection-cpe-232-data-models/test.csv"
sample_path = "/kaggle/input/competitions/fraud-detection-cpe-232-data-models/sample_submission.csv"
```

3. Load Data

โหลดไฟล์ CSV ทั้งสาม (`train.csv`, `test.csv`, `sample_submission.csv`) เข้าสู่ DataFrame และแสดงขนาดของข้อมูลเพื่อตรวจสอบว่าโหลดครบถ้วน

```
print("Loading data...")
train      = pd.read_csv(train_path)
test       = pd.read_csv(test_path)
sample_sub = pd.read_csv(sample_path)
print(f"Train shape: {train.shape}")
print(f"Test  shape: {test.shape}")
```

```
Loading data...
Train shape: (5408227, 12)
Test  shape: (954393, 11)
```

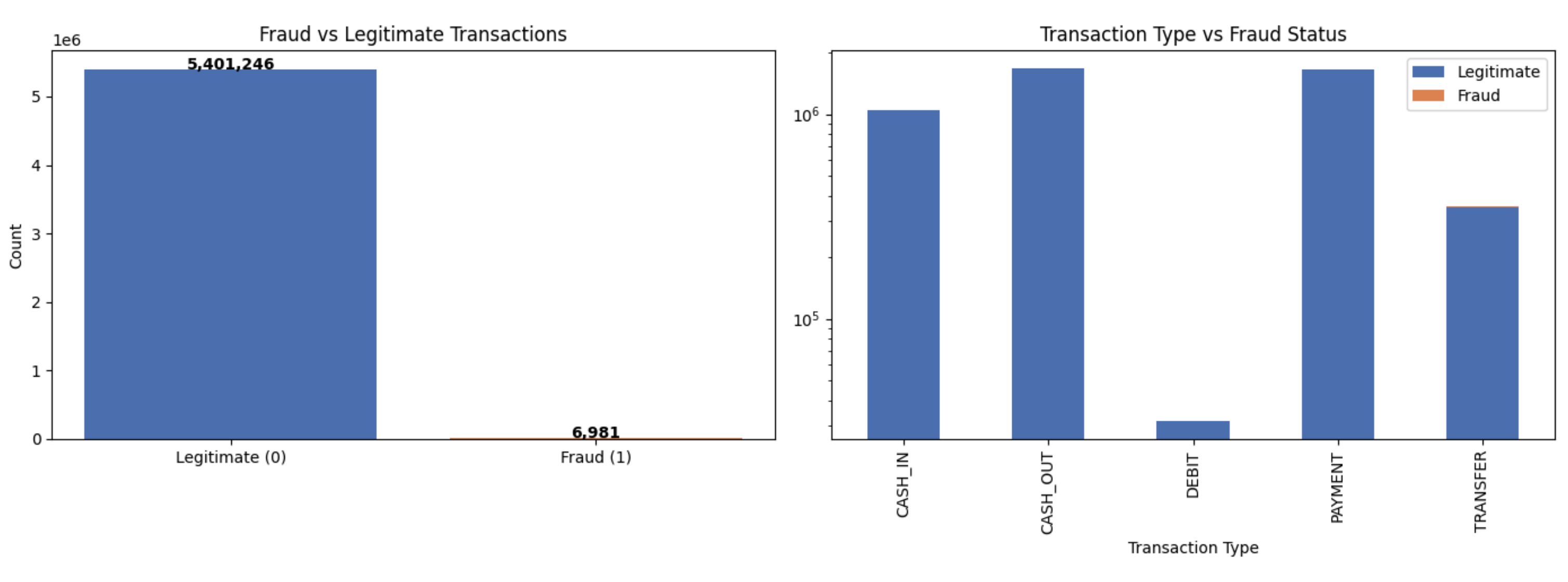
4. Exploratory Data Analysis (EDA)

วิเคราะห์ข้อมูลเบื้องต้นเพื่อทำความเข้าใจโครงสร้างและปัญหาของ Dataset ได้แก่

- **Class Distribution** — ตรวจสอบสัดส่วน Fraud vs. Legitimate เพื่อวางแผนรับมือ Class Imbalance
- **Transaction Type Analysis** — วิเคราะห์ว่า Fraud เกิดขึ้นในประเภทธุรกรรมใดบ้าง
- **Missing Value Check** — ตรวจสอบคอลัมน์ที่มีค่าว่าง (`dst_bal`, `dst_new_bal` สำหรับ Merchant)

```
print("=== Dataset Overview ===")
print("Train shape:", train.shape)
print()
print("is_fraud distribution:")
print(train['is_fraud'].value_counts())
print(f"Fraud rate: {train['is_fraud'].mean():.6f}")
print()
print("Transaction types:")
print(train['transac_type'].value_counts())
print()
print("Missing values:")
print(train.isnull().sum())
print()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ax1 = axes[0]
counts = train['is_fraud'].value_counts()
ax1.bar(['Legitimate (0)', 'Fraud (1)'], counts.values, color=['#4c72b0', '#dd8452'])
ax1.set_title('Fraud vs Legitimate Transactions')
ax1.set_ylabel('Count')
for i, v in enumerate(counts.values):
    ax1.text(i, v + 1000, f'{v:,}', ha='center', fontweight='bold')

ax2 = axes[1]
ct = pd.crosstab(train['transac_type'], train['is_fraud'])
ct.plot(kind='bar', stacked=True, ax=ax2, color=['#4c72b0', '#dd8452'])
ax2.set_title('Transaction Type vs Fraud Status')
ax2.set_xlabel('Transaction Type')
ax2.set_yscale('log')
ax2.legend(['Legitimate', 'Fraud'])
plt.tight_layout()
plt.show()
```

```
=== Dataset Overview ===
Train shape: (5408227, 12)

is_fraud distribution:
is_fraud
0    5401246
1       6981
Name: count, dtype: int64
Fraud rate: 0.001291

Transaction types:
transac_type
CASH_OUT    1677793
PAYMENT     1646545
CASH_IN     1050727
TRANSFER     353231
DEBIT         31600
Name: count, dtype: int64

Missing values:
id                       0
time_ind                 0
transac_type        648331
amount              702632
src_acc                  0
src_bal             269799
src_new_bal              0
dst_acc                  0
dst_bal             270191
dst_new_bal              0
is_fraud                 0
is_flagged_fraud         0
dtype: int64
```



5. Feature Engineering

สร้าง Feature ใหม่จากความรู้เชิงโดเมน (Domain Knowledge) โดยใช้เทคนิคต่าง ๆ ดังนี้

- **Data Imputation** — เติมค่าว่างใน `dst_bal` และ `dst_new_bal` ด้วย 0 (Merchant ไม่รายงาน Balance)
- **Accounting Error Features** — คำนวณความคลาดเคลื่อนของ Balance ด้วยสมการบัญชี (`src_new_bal + amount - src_bal`) หาก Balance ไม่สมดุลแสดงว่ามีความผิดปกติ
- **Zero-Balance Flags** — สร้าง Binary Flag เพื่อตรวจจับ Pattern การโอนเงินจนบัญชีกลายเป็น 0 ซึ่งเป็น Pattern หลักของ Fraud
- **Risky Transaction Type Flag** — Fraud ใน Dataset นี้เกิดขึ้นเฉพาะใน `TRANSFER` และ `CASH-OUT` เท่านั้น
- **Ratio Features** — คำนวณสัดส่วนของ Amount เทียบกับ Balance เพื่อจับธุรกรรมที่ขนาดผิดปกติ
- **Log Transforms** — ใช้ `log1p()` เพื่อลด Skewness ของตัวเลขขนาดใหญ่
- **Time Features** — แยก `hour_of_day` และ `day_of_month` จาก `time_ind`
- **Account-Level Aggregation** — สร้าง Feature ระดับบัญชี เช่น จำนวนครั้ง, ยอดรวม, และความถี่ธุรกรรมเสี่ยง เพื่อจับ Fraudster ที่ทำซ้ำหลายรายการ

```
def compute_src_acc_stats(train_raw):
    """
    Compute per-source-account aggregate stats from raw training data.
    Computed once from the full training set to avoid test-set leakage.
    """
    risky_types = {'TRANSFER', 'CASH_OUT', 'CASH-OUT'}
    g = train_raw.groupby('src_acc')
    stats = pd.DataFrame({
        'src_txn_count': g['amount'].count(),
        'src_amt_mean':  g['amount'].mean(),
        'src_amt_total': g['amount'].sum(),
        'src_amt_max':   g['amount'].max(),
        'src_fq_risky':  train_raw.assign(
            _risky=train_raw['transac_type'].isin(risky_types).astype(int)
        ).groupby('src_acc')['_risky'].mean(),
    }).reset_index()
    return stats


def preprocess_data(df, src_acc_stats=None, is_train=True):
    """
    Full feature engineering pipeline.
    src_acc_stats: pre-computed per-account aggregate stats from training set.
    """
    df = df.copy()

    # --- 5A. Handle missing destination balances (merchants) ---
    df['is_merchant_dst'] = df['dst_bal'].isna().astype(np.int8)
    df['dst_bal']         = df['dst_bal'].fillna(0)
    df['dst_new_bal']     = df['dst_new_bal'].fillna(0)

    # --- 5B. Accounting Error Features ---
    # Fraud often creates a mismatch in the balance equation
    df['error_orig_bal'] = df['src_new_bal'] + df['amount'] - df['src_bal']
    df['error_dest_bal'] = df['dst_bal']     + df['amount'] - df['dst_new_bal']

    df['flag_orig_mismatch'] = (df['error_orig_bal'].abs() > 0.01).astype(np.int8)
    df['flag_dest_mismatch'] = (df['error_dest_bal'].abs() > 0.01).astype(np.int8)

    # --- 5C. Zero-Balance Indicators ---
    # Classic fraud pattern: source account drained completely to zero
    df['src_bal_zero_before'] = (df['src_bal']     == 0).astype(np.int8)
    df['src_bal_zero_after']  = (df['src_new_bal'] == 0).astype(np.int8)
    df['dst_bal_zero_before'] = (df['dst_bal']     == 0).astype(np.int8)
    df['dst_bal_zero_after']  = (df['dst_new_bal'] == 0).astype(np.int8)

    df['flag_src_drained'] = (
        (df['src_bal'] > 0) & (df['src_new_bal'] == 0)
    ).astype(np.int8)

    # --- 5D. Transaction Type Risk Flag ---
    # Fraud in PaySim ONLY occurs in TRANSFER and CASH_OUT
    risky_types = {'TRANSFER', 'CASH_OUT', 'CASH-OUT'}
    df['is_risky_type'] = df['transac_type'].isin(risky_types).astype(np.int8)

    # --- 5E. Amount & Balance Ratio Features ---
    df['amount_to_src_bal_ratio'] = df['amount'] / (df['src_bal']     + 1)
    df['amount_to_dst_bal_ratio'] = df['amount'] / (df['dst_bal']     + 1)
    df['src_bal_change']          = df['src_new_bal'] - df['src_bal']
    df['dst_bal_change']          = df['dst_new_bal'] - df['dst_bal']

    # --- 5F. Log Transforms (reduce skewness) ---
    df['amount_log']      = np.log1p(df['amount'])
    df['src_bal_log']     = np.log1p(df['src_bal'])
    df['dst_bal_log']     = np.log1p(df['dst_bal'])
    df['src_new_bal_log'] = np.log1p(df['src_new_bal'])
    df['dst_new_bal_log'] = np.log1p(df['dst_new_bal'])

    # --- 5G. Time Features ---
    df['hour_of_day']  = df['time_ind'] % 24
    df['day_of_month'] = (df['time_ind'] // 24) % 31

    # --- 5H. Account-Level Aggregation Features (from train stats) ---
    if src_acc_stats is not None:
        df = df.merge(src_acc_stats, on='src_acc', how='left')
        df['src_txn_count'] = df['src_txn_count'].fillna(1)
        df['src_amt_mean']  = df['src_amt_mean'].fillna(df['amount'])
        df['src_amt_total'] = df['src_amt_total'].fillna(df['amount'])
        df['src_amt_max']   = df['src_amt_max'].fillna(df['amount'])
        df['src_fq_risky']  = df['src_fq_risky'].fillna(0)

    # --- 5I. One-Hot Encode Transaction Type ---
    df = pd.get_dummies(df, columns=['transac_type'], drop_first=False)

    # --- 5J. Drop Identifiers ---
    cols_to_drop = ['src_acc', 'dst_acc', 'time_ind']
    if 'id' in df.columns:
        cols_to_drop.append('id')
    df.drop(columns=[c for c in cols_to_drop if c in df.columns], inplace=True)

    return df


print("Computing account-level stats from training data...")
src_acc_stats = compute_src_acc_stats(train)

print("Preprocessing training data...")
X      = preprocess_data(train.drop(columns=['is_fraud']), src_acc_stats=src_acc_stats, is_train=True)
y      = train['is_fraud'].values

print("Preprocessing test data...")
X_test = preprocess_data(test, src_acc_stats=src_acc_stats, is_train=False)

X, X_test = X.align(X_test, join='left', axis=1, fill_value=0)
print(f"Feature count: {X.shape[1]}")
print("Features:", X.columns.tolist())
```

```
Computing account-level stats from training data...
Preprocessing training data...
Preprocessing test data...
Feature count: 38
Features: ['amount', 'src_bal', 'src_new_bal', 'dst_bal', 'dst_new_bal', 'is_flagged_fraud', 'is_merchant_dst', 'error_orig_bal', 'error_dest_bal', 'flag_orig_
mismatch', 'flag_dest_mismatch', 'src_bal_zero_before', 'src_bal_zero_after', 'dst_bal_zero_before', 'dst_bal_zero_after', 'flag_src_drained', 'is_risky_type',
'amount_to_src_bal_ratio', 'amount_to_dst_bal_ratio', 'src_bal_change', 'dst_bal_change', 'amount_log', 'src_bal_log', 'dst_bal_log', 'src_new_bal_log', 'dst_n
ew_bal_log', 'hour_of_day', 'day_of_month', 'src_txn_count', 'src_amt_mean', 'src_amt_total', 'src_amt_max', 'src_fq_risky', 'transac_type_CASH_IN', 'transac_t
ype_CASH_OUT', 'transac_type_DEBIT', 'transac_type_PAYMENT', 'transac_type_TRANSFER']
```

6. Model Training — XGBoost + Random Forest (5-Fold CV)

ฝึกโมเดลสองตัวแบบขนาน โดยใช้ **Stratified K-Fold Cross-Validation (5 Folds)** เพื่อให้ทุก Fold มีสัดส่วน Fraud เท่ากัน และเก็บ Out-of-Fold (OOF) Predictions สำหรับการ Ensemble

- **XGBoost** — Gradient Boosting บน Decision Tree ใช้ `scale_pos_weight` เพื่อจัดการ Class Imbalance, `eval_metric='aucpr'` (Area Under Precision-Recall Curve) เหมาะกับข้อมูลที่ไม่สมดุล และ `early_stopping_rounds=100` เพื่อหยุดเมื่อ Validation Score ไม่ดีขึ้น
- **Random Forest** — Ensemble ของ Decision Tree แบบ Bagging ใช้ `class_weight='balanced'` เพื่อรับมือ Class Imbalance และ `max_features='sqrt'` เพื่อลด Correlation ระหว่าง Tree

```
n_neg = (y == 0).sum()
n_pos = (y == 1).sum()
scale_pos_weight = n_neg / n_pos
print(f"Positive: {n_pos}  |  Negative: {n_neg}  |  scale_pos_weight: {scale_pos_weight:.2f}")

N_FOLDS  = 5
kf       = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=42)

oof_xgb  = np.zeros(len(X))
oof_rf   = np.zeros(len(X))
test_xgb = np.zeros(len(X_test))
test_rf  = np.zeros(len(X_test))

# ── XGBoost ──────────────────────────────────────────────────
print("\n========== XGBoost Training ==========")
for fold, (tr_idx, val_idx) in enumerate(kf.split(X, y)):
    print(f"--- XGB Fold {fold + 1}/{N_FOLDS} ---")
    X_tr, y_tr   = X.iloc[tr_idx], y[tr_idx]
    X_val, y_val = X.iloc[val_idx], y[val_idx]

    # early_stopping_rounds in constructor (XGBoost >= 2.0 requirement)
    model_xgb = xgb.XGBClassifier(
        n_estimators          = 2000,
        learning_rate         = 0.05,
        max_depth             = 7,
        subsample             = 0.85,
        colsample_bytree      = 0.85,
        min_child_weight      = 5,
        reg_alpha             = 0.2,
        reg_lambda            = 1.0,
        gamma                 = 0.1,
        scale_pos_weight      = scale_pos_weight,
        tree_method           = 'hist',
        eval_metric           = 'aucpr',
        early_stopping_rounds = 100,
        random_state          = 42,
        n_jobs                = -1,
    )
    model_xgb.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=100)
    oof_xgb[val_idx] += model_xgb.predict_proba(X_val)[:, 1]
    test_xgb         += model_xgb.predict_proba(X_test)[:, 1] / N_FOLDS

xgb_oof_f1 = f1_score(y, (oof_xgb >= 0.5).astype(int), average='macro')
print(f"XGBoost OOF F1 Macro (threshold=0.5): {xgb_oof_f1:.5f}")

# ── Random Forest ─────────────────────────────────────────────
print("\n========== Random Forest Training ==========")
for fold, (tr_idx, val_idx) in enumerate(kf.split(X, y)):
    print(f"--- RF Fold {fold + 1}/{N_FOLDS} ---")
    X_tr, y_tr   = X.iloc[tr_idx], y[tr_idx]
    X_val, y_val = X.iloc[val_idx], y[val_idx]

    model_rf = RandomForestClassifier(
        n_estimators     = 300,
        max_depth        = 25,
        min_samples_leaf = 3,
        max_features     = 'sqrt',
        class_weight     = 'balanced',
        n_jobs           = -1,
        random_state     = 42,
    )
    model_rf.fit(X_tr, y_tr)
    oof_rf[val_idx] += model_rf.predict_proba(X_val)[:, 1]
    test_rf         += model_rf.predict_proba(X_test)[:, 1] / N_FOLDS

rf_oof_f1 = f1_score(y, (oof_rf >= 0.5).astype(int), average='macro')
print(f"Random Forest OOF F1 Macro (threshold=0.5): {rf_oof_f1:.5f}")
```

```
Positive: 6981  |  Negative: 5401246  |  scale_pos_weight: 773.71

========== XGBoost Training ==========
--- XGB Fold 1/5 ---
[0]	validation_0-aucpr:0.90645
[100]	validation_0-aucpr:0.97715
[200]	validation_0-aucpr:0.98033
[300]	validation_0-aucpr:0.98057
[400]	validation_0-aucpr:0.98069
[500]	validation_0-aucpr:0.98072
[587]	validation_0-aucpr:0.98073
--- XGB Fold 2/5 ---
[0]	validation_0-aucpr:0.89816
[100]	validation_0-aucpr:0.97978
[200]	validation_0-aucpr:0.98393
[300]	validation_0-aucpr:0.98496
[400]	validation_0-aucpr:0.98573
[500]	validation_0-aucpr:0.98603
[569]	validation_0-aucpr:0.98593
--- XGB Fold 3/5 ---
[0]	validation_0-aucpr:0.85543
[100]	validation_0-aucpr:0.97376
[200]	validation_0-aucpr:0.97763
[300]	validation_0-aucpr:0.97856
[400]	validation_0-aucpr:0.97925
[500]	validation_0-aucpr:0.97986
[600]	validation_0-aucpr:0.98018
[700]	validation_0-aucpr:0.98063
[800]	validation_0-aucpr:0.98053
[900]	validation_0-aucpr:0.98084
[1000]	validation_0-aucpr:0.98093
[1100]	validation_0-aucpr:0.98076
[1121]	validation_0-aucpr:0.98075
--- XGB Fold 4/5 ---
[0]	validation_0-aucpr:0.86811
[100]	validation_0-aucpr:0.97789
[200]	validation_0-aucpr:0.98468
[300]	validation_0-aucpr:0.98657
[400]	validation_0-aucpr:0.98733
[500]	validation_0-aucpr:0.98765
[600]	validation_0-aucpr:0.98774
[629]	validation_0-aucpr:0.98775
--- XGB Fold 5/5 ---
[0]	validation_0-aucpr:0.90428
[100]	validation_0-aucpr:0.97855
[200]	validation_0-aucpr:0.98295
[300]	validation_0-aucpr:0.98312
[400]	validation_0-aucpr:0.98293
[416]	validation_0-aucpr:0.98294
XGBoost OOF F1 Macro (threshold=0.5): 0.94544

========== Random Forest Training ==========
--- RF Fold 1/5 ---
--- RF Fold 2/5 ---
--- RF Fold 3/5 ---
--- RF Fold 4/5 ---
--- RF Fold 5/5 ---
Random Forest OOF F1 Macro (threshold=0.5): 0.98198
```

7. Ensemble & Threshold Optimization

รวมผลจากทั้งสองโมเดลและหา Decision Threshold ที่เหมาะสมที่สุด

- **Weighted Soft Voting Ensemble** — เฉลี่ย Predicted Probability แบบถ่วงน้ำหนัก (XGBoost 65% + Random Forest 35%) เนื่องจาก XGBoost มักแม่นยำกว่าบน Tabular Data
- **Fine-Grained Threshold Search** — ค้นหา Threshold ที่เหมาะสมที่สุดในช่วง 0.001–0.999 ด้วย Step Size 0.001
- โดยวัดจาก OOF F1 Macro Score เพื่อหลีกเลี่ยง Data Leakage

```
# Weighted average — XGBoost 65%, Random Forest 35%
W_XGB = 0.65
W_RF  = 0.35

oof_ensemble  = W_XGB * oof_xgb  + W_RF * oof_rf
test_ensemble = W_XGB * test_xgb + W_RF * test_rf

# Fine-grained threshold search (step = 0.001)
print("Optimizing decision threshold for F1 Macro...")
best_thresh = 0.5
best_f1     = 0.0

for t in np.arange(0.001, 0.999, 0.001):
    preds = (oof_ensemble >= t).astype(int)
    score = f1_score(y, preds, average='macro')
    if score > best_f1:
        best_f1     = score
        best_thresh = t

print(f"Best OOF F1 Macro  : {best_f1:.6f}")
print(f"Best Threshold     : {best_thresh:.3f}")
print(f"XGBoost at best t  : {f1_score(y, (oof_xgb >= best_thresh).astype(int), average='macro'):.6f}")
print(f"RF      at best t  : {f1_score(y, (oof_rf  >= best_thresh).astype(int), average='macro'):.6f}")
```

```
Optimizing decision threshold for F1 Macro...
Best OOF F1 Macro  : 0.983883
Best Threshold     : 0.786
XGBoost at best t  : 0.973391
RF      at best t  : 0.979332
```

8. Export Submission

นำ Threshold ที่ดีที่สุดจากขั้นตอนก่อนหน้ามาแปลง Probability เป็น Binary Prediction (0/1) แล้วบันทึกเป็นไฟล์ `submission_FTD.csv` เพื่อใช้สำหรับการ submission

```
final_preds = (test_ensemble >= best_thresh).astype(int)
sample_sub['is_fraud'] = final_preds
sample_sub.to_csv("submission_FTD.csv", index=False)
print(f"Saved submission.csv  —  fraud predicted: {final_preds.sum():,} / {len(final_preds):,}")
print("Done. Submit submission_FTD.csv to Kaggle.")
```

```
Saved submission.csv  —  fraud predicted: 1,193 / 954,393
Done. Submit submission_FTD.csv to Kaggle.
```